

# 2026학년도 2학기 언어데이터과학

## 제7강 Python 활용 (2) 데이터프레임과 시각화

박수민

서울대학교 인문대학 언어학과

2026년 9월 23일 수요일

## 지난 시간에 배운 것

- 1 배열은 **한 자료형**만 담는 대신 속도와 짧은 코드를 준다.
- 2 벡터화 연산과 불리언 마스크로 for문 없이 계산했다.
- 3 5강에서 AWK로 구한 값을 numpy로 모두 다시 구했다.

## 오늘의 목표

- 1 데이터프레임으로 **수치 열과 문자 열이 섞인 표**를 다룰 수 있다.
- 2 고르기·정렬·새 열 만들기·붙이기·뭉기를 할 수 있다.
- 3 결측값이 무엇인지 알고 처리할 수 있다.
- 4 seaborn으로 분포와 관계를 그리고 파일로 남길 수 있다.

# 오늘의 출발점

## 6강 마지막 슬라이드에 적어 둔 코드

```
import pandas as pd

df = pd.read_csv('data/apicius/index.tsv', sep='\t')
print(df.head())
print(df['words'].describe())
```

## 오늘 할 일

6강에서는 index.tsv 한 표를 numbers, titles, words 배열 셋으로 쪼개 들고, 셋의 순서가 맞기를 빌면서 썼다. 오늘은 표를 **표인 채로** 다룬다. 그리고 5강이 미뤄 둔 그림을 드디어 그린다.

# 사전 준비

## 터미널 입력 (Codespaces)

```
cd /workspaces/LDS2026  
pip install pandas seaborn
```

## 노트북 07-pandas.ipynb의 첫 셀

```
import pandas as pd  
import seaborn as sns  
import matplotlib.pyplot as plt  
  
sns.set_theme(style='whitegrid')
```

pd와 sns도 np처럼 전 세계가 지키는 약칭이다. seaborn을 깔면 pandas와 matplotlib이 함께 달려 온다.

# 1 데이터프레임

표를 표인 채로  
열과 행 고르기

## 2 표 다루기

## 3 시각화

## 4 마무리

# 6강이 남긴 불편

6강 — 배열 셋

|         |      |       |      |
|---------|------|-------|------|
| numbers | 1    | 2     | 3    |
| titles  | WINE | HONEY | VERM |
| words   | 144  | 82    | 73   |

셋의 순서가 맞는다는 것은 우리가 기억할 뿐이다. 하나만 정렬해도 약속이 깨진다

⇒

7강 — 데이터프레임 하나

|    | number | title | words |
|----|--------|-------|-------|
| df | 1      | WINE  | 144   |
|    | 2      | HONEY | 82    |
|    | 3      | VERM  | 73    |

한 이름 아래 함께 움직인다.  
열마다 자료형이 달라도 된다

**pandas:** 표(table)를 다루는 라이브러리. 이름은 *panel data*에서 왔다.

## Series — 색인이 붙은 배열

### 노트북 입력

```
s = pd.Series([144, 82, 73],
               index=['가', '나', '다'])
print(s)
print(s['나'])
```

### 실행 결과

```
가    144
나     82
다     73
dtype: int64
82
```

### 배열 + 이름표

**Series**는 numpy 배열 하나에 **색인(index)**과 **이름**을 붙인 것이다.

- 자리(0, 1, 2)로도 부르고 이름('나')으로도 부른다.
- dtype은 그대로 있다. 한 Series에는 여전히 한 자료형만 담긴다.

# DataFrame — 색인을 공유하는 Series의 모음

|    |   | number | title | words | 열 이름         |
|----|---|--------|-------|-------|--------------|
| 색인 | 0 | 1      | WINE  | 144   | 열 하나가 Series |
|    | 1 | 2      | HONEY | 82    |              |
|    | 2 | 3      | VERM  | 73    |              |

행 하나가 레시피 하나

두 가지를 함께 기억한다

색인은 모든 열이 함께 쓴다. 그래서 어느 열을 정렬해도 행이 흩어지지 않는다.

# 파일에서 읽기

## 노트북 입력

```
df = pd.read_csv('data/apicius/index.tsv', sep='\t')
print(df.shape)
print(df.columns)
```

## 실행 결과

```
(499, 3)
Index(['number', 'title', 'words'], dtype='str')
```

**read\_csv** 이름은 CSV지만 sep만 바꾸면 어떤 구분자든 읽는다  
**sep='\t'** 탭으로 나눈다. 6강 np.loadtxt의 delimiter  
**shape** (행, 열). 머리글은 **저절로** 열 이름이 되었다

# 노트북은 표를 표로 그려 준다

셀의 마지막 식이 `df.head(3)` 일 때 — 여러분의 화면

|   | number | title                         | words |
|---|--------|-------------------------------|-------|
| 0 | 1      | FINE SPICED WINE              | 144   |
| 1 | 2      | HONEY REFRESHER FOR TRAVELERS | 82    |
| 2 | 3      | ROMAN VERMOUTH                | 73    |

`print(df.head(3))` 일 때 — 이 슬라이드. 6강에서 정한 그 약속이다

|   | number | title                         | words |
|---|--------|-------------------------------|-------|
| 0 | 1      | FINE SPICED WINE              | 144   |
| 1 | 2      | HONEY REFRESHER FOR TRAVELERS | 82    |
| 2 | 3      | ROMAN VERMOUTH                | 73    |

# info() – 표의 신상명세

## 노트북 입력

```
df.info()
```

## 실행 결과

```
<class 'pandas.DataFrame'>
RangeIndex: 499 entries, 0 to 498
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0   number  499 non-null    int64  
 1   title   498 non-null    str     
 2   words   499 non-null    int64  
dtypes: int64(2), str(1)
```

info는 **얼마다** 자료형이 다르다는 것을 보여 준다. 6강의 배열로는 못 하던 일이다.

# title만 498이다

Non-Null Count가 알려 준 것

499행인데 title은 498개뿐이다. 어딘가 한 자리가 **비어 있다**.

6강은 이것을 못 봤다

- `np.loadtxt(..., dtype=str)`는 빈 칸을 빈 문자열 `' '`로 읽는다. 값이 있는 것처럼 보이니 아무도 눈치채지 못한다.
- `pandas`는 빈 칸을 **결측값**(missing value, NaN)으로 표시해 세어 준다.

어느 레시피인지는 §2에서 찾는다. 지금은 `info()`를 읽는 습관만 챙긴다.

## describe() – 5강과 6강이 한 줄에

### 노트북 입력

```
print(df['words'].describe())
```

### 실행 결과

```
count    499.000000
mean     45.124248
std      34.774035
min       0.000000
25%      20.000000
50%      33.000000
75%      61.000000
max     273.000000
Name: words, dtype: float64
```

5강의 AWK 스크립트 두 개와 6강의 numpy 여섯 줄이 여기 다 들어 있다. std는 **기본이 ddof=1**이다. numpy와 반대이니 조심한다.

# 열 고르기: 대괄호 한 겹과 두 겹

## 노트북 입력

```
print(type(df['words']).__name__)  
print(type(df[['number', 'words']]).__name__)  
print(df[['number', 'words']].head(3))
```

## 실행 결과

```
Series  
DataFrame  
   number  words  
0        1   144  
1        2    82  
2        3    73
```

## 한 겹은 Series, 두 겹은 DataFrame

df['words']는 열 **하나**를 꺼내니 Series다. df[['number', 'words']]의 안쪽 대괄호는 **열 이름의 목록**이고, 목록을 주면 표가 돌아온다. 열 하나짜리 표가 필요하다면 df[['words']]라고 쓴다.

## Series 안에는 6강의 배열이 있다

### 노트북 입력

```
w = df['words']  
print(w.mean(), w.std(), w.median())  
print(w.values[:5])  
print(w[w > 200].values)
```

### 실행 결과

```
45.124248496993985 34.77403524607915 33.0  
[144  82  73  82  23]  
[206 273]
```

**6강에서 배운 것이 그대로 통한다.** 집계 메서드도, 불리언 마스크도 이름만 같은 것이 아니라 **같은 것이다**. `.values`로 언제든지 numpy 배열을 꺼낼 수 있다. 새로 외울 것은 색인뿐이다.

# 행 고르기: loc와 iloc

## 노트북 입력

```
print(df.loc[0, 'title'])  
print(df.iloc[0, 1])  
print(df.loc[0:2, 'words'].values)  
print(df.iloc[0:2, 2].values)
```

## 실행 결과

```
FINE SPICED WINE  
FINE SPICED WINE  
[144  82  73]  
[144  82]
```

**loc** 열은 이름으로, 행은 색인으로. 끝값을 포함해 세 개가 나왔다

**iloc** 둘 다 자리로. 6강의 배열처럼 끝값을 빼서 두 개가 나왔다

지금은 색인이 0, 1, 2, ...라 둘이 거의 같아 보인다. 앞으로 정렬하거나 골라내고 나면 색인이 219, 202처럼 흩어져 둘이 완전히 달라진다.

# 확인 질문

## 여기까지 정리

- 1 Series와 DataFrame의 차이를 “색인”이라는 말로 설명해 보자.
- 2 `df['words']`와 `df[['words']]`는 무엇이 다른가?
- 3 `df.info()`에서 `title`만 498인 것은 무슨 뜻인가?
- 4 `df['words'].describe()`의 `std`는  $n$ 으로 나눈 값인가,  $n - 1$ 인가?
- 5 `df.loc[0:2]`는 몇 행인가? `df.iloc[0:2]`는?

## 1 데이터프레임

## 2 표 다루기

고르고 정렬하기  
열 만들기과 세기  
두 표를 붙이고 묶기

## 3 시각화

## 4 마무리

## 조건으로 고르기 – 이번에는 제목이 달려 온다

### 노트북 입력

```
print(df[df['words'] > 200])
```

### 실행 결과

|     | number |      | title           | words |
|-----|--------|------|-----------------|-------|
| 140 | 141    |      | APICIAN DISH    | 206   |
| 185 | 186    | PEAS | [supreme style] | 273   |

### 6강에서는 두 줄이 필요했다

마스크를 words로 만들어 numbers와 titles에 **따로** 씌워야 했다. 이제 마스크 하나가 **행 전체**를 고른다. 왼쪽에 남은 140, 185가 원래 색인이다.

# 조건 잇기와 query

## 노트북 입력

```
m = df['words'] < df['words'].mean()
print(m.sum(), f'{m.mean():.1%}')
print(len(df.query('20 <= words <= 61')))
```

## 실행 결과

```
313 62.7%
258
```

## &, |, ~와 괄호는 6강 그대로

- `df[(df['words'] >= 20) & (df['words'] <= 61)]` — 익숙한 형태
- `df.query('20 <= words <= 61')` — 조건을 **문자열**로 적는다. 열 이름을 그냥 쓰고, 파이썬처럼 부등호를 이어 쓸 수 있어 읽기 쉽다.

# 정렬해도 약속이 깨지지 않는다

## 노트북 입력

```
print(df.sort_values('words').head(3))
```

## 실행 결과

|     | number |                                                  | title | words |
|-----|--------|--------------------------------------------------|-------|-------|
| 219 | 220    | FOR ROASTS: PEPPER, LOVAGE, CORIANDER, CARRAWAY, | 0     |       |
| 202 | 203    | BEANS SAUTÉ                                      | 6     |       |
| 307 | 308    | FRIED BULBS                                      | 6     |       |

6강에서 `np.sort(words)`를 하면 `titles`는 제자리에 남아 짝이 어긋났다.  
`sort_values`는 **행을 통째로** 옮긴다. 색인 219가 따라온 것이 그 증거다.

# 가장 긴 것 몇 개만

## 노트북 입력

```
print(df.nlargest(4, 'words'))
```

## 실행 결과

|     | number |          | title           | words |
|-----|--------|----------|-----------------|-------|
| 185 | 186    | PEAS     | [supreme style] | 273   |
| 140 | 141    |          | APICIAN DISH    | 206   |
| 173 | 174    | MOVEABLE | APPETIZERS      | 184   |
| 284 | 285    |          | PIG'S PAUNCH    | 177   |

## 6강의 argmax가 하던 일

`sort_values('words', ascending=False).head(4)`와 같은 결과이고, 더 짧다. 반대는 `nsmllest`다.

# 새 열 만들기

노트북 입력 — 없는 이름에 대입하면 열이 새로 생긴다

```
df['z'] = (df['words'] - df['words'].mean()) / df['words'].std()
print(df[df['z'].abs() > 3].round(2))
```

실행 결과 — 숙제03 1번의 답. ...는 화면 폭 탓이지 값이 잘린 것이 아니다

|     | number |                                                   | title                     | words | z    |
|-----|--------|---------------------------------------------------|---------------------------|-------|------|
| 139 | 140    | A RICH ENTRÉE OF FISH, POULTRY AND SAUSAGE IN ... |                           | 174   | 3.71 |
| 140 | 141    |                                                   | APICIAN DISH              | 206   | 4.63 |
| 173 | 174    |                                                   | MOVEABLE APPETIZERS       | 184   | 3.99 |
| 175 | 176    |                                                   | STUFFED PUMPKIN FRITTERS  | 156   | 3.19 |
| 185 | 186    |                                                   | PEAS [supreme style]      | 273   | 6.55 |
| 284 | 285    |                                                   | PIG'S PAUNCH              | 177   | 3.79 |
| 359 | 360    |                                                   | STUFFED BONED KID OR LAMB | 157   | 3.22 |

## 문자열 열 다루기: .str

노트북 입력 — .str를 앞에 붙이면 파이썬 문자열 메서드가 열 전체에 적용된다

```
print(df['title'].str.len().head(3))  
df['first'] = df['title'].str.split().str[0]  
print(df['first'].head(3).values)
```

실행 결과 — 445번의 빈 자리 때문에, 정수가 아니라 float64가 되었다

```
0    16.0  
1    29.0  
2    14.0  
Name: title, dtype: float64  
['FINE' 'HONEY' 'ROMAN']
```

정수 열에는 NaN을 담을 수 없다. 6강이 for를 다시 꺼내야 했던 자리이기도 하다 —  
[len(w) for w in vocab]이 .str.len()이 되었다.

## value\_counts – 무엇이 몇 번인가

### 노트북 입력

```
vc = df['first'].value_counts()  
print(vc.head(5))
```

### 실행 결과

```
first  
ANOTHER    116  
SAUCE       37  
A           25  
TO          23  
BOILED      14  
Name: count, dtype: int64
```

### 레시피 499편 중 116편이 “ANOTHER”로 시작한다

3강의 `sort | uniq -c | sort -nr`가 메서드 하나가 되었다. 가장 흔한 제목은 ANOTHER WAY로, 그대로 **29번** 나온다. 제목이 앞 레시피에 기대어 있다는 뜻이고, 이런 문서 구조는 8강에서 다시 이야기한다.

## awk/book.awk: 레시피가 몇 권에 있는가

```
1 BEGIN {
2     RS = "\n\n\n+"
3     FS = "\n\n"
4     OFS = "\t"
5     print "number", "book"                # 머리글
6 }
7 $1 ~ /^BOOK [IVX]+\./ {                  # 'BOOK V. LEGUMES' 같은 권 제목
8     split($1, line, "\n")
9     book = line[1]
10    sub(/^BOOK /, "", book)                # -> 'V. LEGUMES'
11    sub(/ \[[0-9]+\]/, "", book)           # 각주 표시 떼기
12 }
13 $1 ~ /^\[ [0-9]+\]/ {                    # 4강 index.awk 와 같은 규칙
14     match($1, /[0-9]+/)
15     print substr($1, RSTART, RLENGTH), book
16 }
```

# 만들어 보기

## 터미널 입력

```
awk -f awk/book.awk data/apicius/recipes-lf.txt > data/apicius/book.tsv  
cut -f2 data/apicius/book.tsv | uniq -c | head -n 4
```

## 실행 결과

```
1 book  
40  
25 II. MINCES  
59 III. THE GARDENER
```

## 4강의 index.awk에서 달라진 곳은 규칙 하나뿐이다

권 제목을 만나면 변수 book에 적어 두고, 그다음 레시피들에 그 값을 붙인다. AWK는 파일을 위에서 아래로 한 번 훑으므로 이것으로 충분하다. 그런데 40편은 권이 비어 있다.

# merge — 열쇠를 맞추어 붙이기

## 노트북 입력

```
bk = pd.read_csv('data/apicius/book.tsv', sep='\t')
df = df.merge(bk, on='number')
print(df.shape)
print(df[['number', 'words', 'book']].tail(3))
```

## 실행 결과

```
(499, 6)
   number  words      book
496    497    47  X. THE FISHERMAN
497    498    29  X. THE FISHERMAN
498    499    25  X. THE FISHERMAN
```

`on='number'`: 두 표에 함께 있는 `number` 열을 **열쇠**로 삼아 행끼리 짝지었다. `numpy`로는 손으로 해야 하는 일이다.

# 결측값 NaN

## 노트북 입력

```
print(df.isna().sum())  
m = df['title'].isna()  
print(df.loc[m, ['number', 'words']])
```

## 실행 결과

```
number    0  
title      1  
words      0  
z          0  
first      1  
book       40  
dtype: int64  
      number  words  
444      445     12
```

## 빈 자리에도 까닭이 있다

445번은 원문에 정말 제목이 없다. 40편의 빈 book은 우리 탓이다. 3강에서 본문을 자를 때 1권 제목 줄이 잘려 나갔다.

# fillna로 메우기

## 노트북 입력

```
df['book'] = df['book'].fillna('I. THE CAREFUL EXPERIENCED COOK')  
print(df['book'].value_counts().head(4))
```

## 실행 결과

```
book  
VII. SUMPTUOUS DISHES      78  
VIII. QUADRUPEDS          68  
X. THE FISHERMAN           68  
III. THE GARDENER          59  
Name: count, dtype: int64
```

**메울 때는 근거가 있어야 한다.** 1권이라는 것을 book.txt에서 확인했기에 메운 것이다.  
근거가 없으면 dropna()로 빼거나 NaN인 채로 두고 그렇게 보고한다.

# groupby – 나누어 보기

```
g = df.groupby('book')['words'].mean()  
print(g.sort_values().round(1))
```

실행 결과 – “평균 45.12단어”는 이 열 개를 뭉겐 값이었다

```
book  
IX. SEAFOOD                28.9  
III. THE GARDENER          31.3  
VII. SUMPTUOUS DISHES      35.2  
X. THE FISHERMAN           35.8  
I. THE CAREFUL EXPERIENCED COOK 42.4  
VI. FOWL                   47.6  
VIII. QUADRUPEDS           48.9  
II. MINCES                 52.1  
V. LEGUMES                 59.6  
IV. MISCELLANEA            81.3  
Name: words, dtype: float64
```

# 묶어 보니 보이는 것

## 짧은 쪽

- IX. SEAFOOD — 28.9단어
- III. THE GARDENER — 31.3단어

재료 하나를 손질하는 법. 문장이 짧고 단계가 적다.

## 긴 쪽

- IV. MISCELLANEA — 81.3단어
- V. LEGUMES — 59.6단어

여러 재료를 층층이 쌓는 요리. 소스와 마무리까지 적는다.

## 5강의 그 말

“요약값 하나는 반드시 무엇인가를 감춘다.” 감춰져 있던 것이 **세 배 차이**였다. 무엇으로 나눌지를 아는 것은 도구가 아니라 **자료를 아는 사람**의 몫이다.

## agg — 한 번에 여러 값

### 노트북 입력

```
print(df.groupby('book')['words']  
      .agg(['count', 'mean', 'median']).round(1).head(3))
```

### 실행 결과

|                                 | count | mean | median |
|---------------------------------|-------|------|--------|
| book                            |       |      |        |
| I. THE CAREFUL EXPERIENCED COOK | 40    | 42.4 | 31.5   |
| II. MINCES                      | 25    | 52.1 | 50.0   |
| III. THE GARDENER               | 59    | 31.3 | 27.0   |

**평균과 중앙값을 나란히 놓는 이유:** 1권은 42.4와 31.5로 벌어져 있고 2권은 52.1과 50.0으로 거의 같다. 5강에서 배운 대로 앞쪽에는 **오른쪽으로 긴 꼬리**가 있다는 뜻이다. 같은 “평균”이라도 사정이 다르다. 4강의 `count[$1]++`가 한 줄이 되었다.

# 확인 질문

## 여기까지 정리

- 1 `df[df['words'] > 200]` 이 6강의 마스크와 다른 점은?
- 2 `sort_values`가 `np.sort`보다 안전한 이유는?
- 3 `merge(bk, on='number')`에서 `on`은 무엇을 정하는가?
- 4 `NaN`을 0으로 채우면 안 되는 이유를 445번 레시피로 설명해 보자.
- 5 `groupby('book')['words'].mean()`은 값을 몇 개 돌려주는가?

## 1 데이터프레임

## 2 표 다루기

## 3 시각화

왜, 그리고 무엇으로  
분포 그리기  
관계 그리기  
그림 남기기  
저장소에 남기기

## 4 마무리

# 수는 감추고 그림은 보여 준다

## 5강에서 미뤄 둔 숙제

“평균 45.12, 중앙값 33, 최댓값 273”을 들으면 어떤 모양이 떠오르는가? 기술통계량 열 개보다 그림 한 장이 빠른 자리가 있다.

## 오늘 그릴 네 가지

**분포** 값이 어디에 몰려 있나 — 히스토그램

**요약** 가운데와 바깥은 어디인가 — 상자그림

**비교** 집단끼리 다른가 — 집단별 상자그림

**관계** 두 변수가 함께 움직이나 — 산점도

그림은 답이 아니라 **질문을 찾는 도구**다. 그린 뒤에는 반드시 수로 확인한다.

# matplotlib과 seaborn

DataFrame — 열에 **이름**이 있으니  
축 이름과 범례를 저절로 붙일 수 있다



seaborn — “이 표의 이 열을 이렇게 그려라”



matplotlib — 선과 점과 글자를 찍는 도구.  
무엇이든 그릴 수 있지만 손이 많이 간다

열 이름을 넘긴다

대신 불러 준다

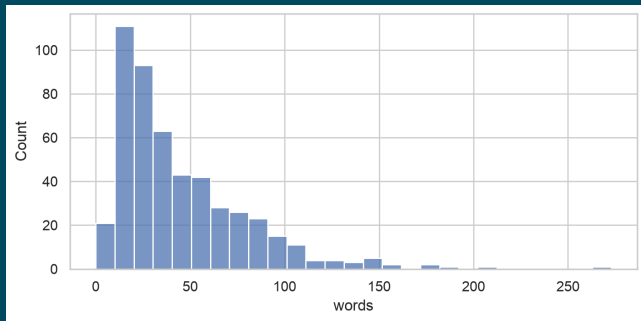
그래서 순서가 이렇다

**표를 먼저 만들고** (\$1, \$2) 그린다. 그리기 좋은 모양으로 표를 정리하는 것이 일의 절반이다. seaborn이 못 하는 세밀한 손질은 matplotlib으로 마무리한다.

# 히스토그램 — 5강이 미뤄 둔 그림

노트북 입력 — 첫 인자가 표, x가 열 이름. 축 이름은 저절로 붙는다

```
sns.histplot(df, x='words')
```

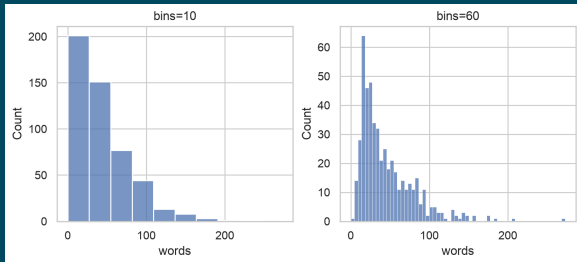


왼쪽에 몰리고 오른쪽으로 길게 끌리는 모양 — 평균 45.12와 중앙값 33의 차이다.

# bins: 몇 칸으로 나눌 것인가

노트북 입력 — 한 줄씩 **따로** 실행한다. 한 셀에 두면 겹쳐 그려진다

```
sns.histplot(df, x='words', bins=10)  
sns.histplot(df, x='words', bins=60)
```

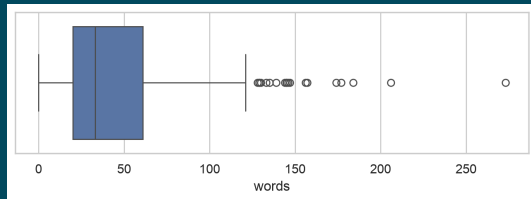


칸이 너무 적으면 뭉개지고, 너무 많으면 들쭉날쭉해진다. 같은 자료로 다른 인상을 줄 수 있으니, 논문에 실을 때는 bins를 적어 둔다.

# 상자그림 – 다섯 수치 요약을 그림으로

## 노트북 입력

```
sns.boxplot(df, x='words')
```



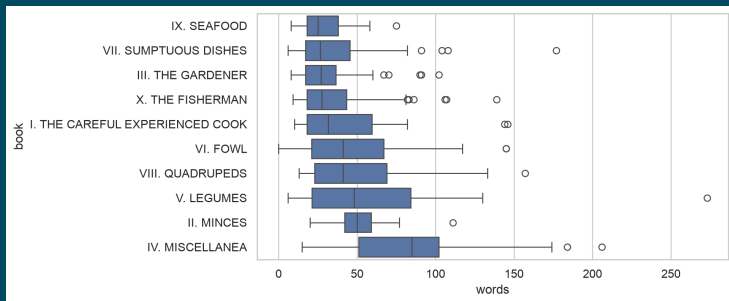
5강의 0 / 20 / 33 / 61 / 273이 여기 다 있다

상자의 양 끝이 Q1(20)과 Q3(61), 가운데 선이 중앙값(33)이다. 수염은 상자 폭의 1.5배까지 뻗고, 그 바깥의 점을 **이상치**로 따로 찍는다. 오른쪽 끝의 외딴 점이 186번 PEAS다.

# 집단별로 나누어 그리기

노트북 입력 — 중앙값 순으로 세운다

```
order = df.groupby('book')['words'].median().sort_values().index  
sns.boxplot(df, x='words', y='book', order=order)
```



# 이 그림이 말하는 것

## 읽는 법

- y에 범주 열을 주면 집단마다 상자 하나
- order로 순서를 정한다. 자모순이 아니라 **값의 순서**로
- 가로로 눕히면 긴 이름표가 들어간다

## 보이는 것

- IV권의 상자는 통째로 오른쪽에 있다
- 대부분의 권은 상자가 겹친다
- 이상치는 어느 권에나 있다

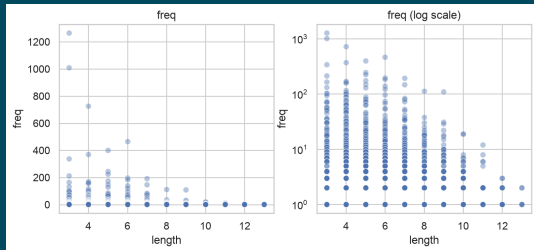
## 앞의 표와 이 그림은 같은 자료다

표는 “IV권 81.3, IX권 28.9”라고 말하고, 그림은 **그 차이가 흩어짐보다 큰지**까지 보여 준다. 권끼리 정말 다른지를 통계적으로 따지는 일은 이 수업의 범위를 넘지만, 물음은 여기서 시작된다.

# 산점도 — 5강의 $r = -0.14$ 를 눈으로

## 노트북 입력

```
wf = pd.read_csv('data/apicius/word-freq.txt', sep=r'\s+',  
                 names=['freq', 'word'])  
wf['length'] = wf['word'].str.len()  
sns.scatterplot(wf, x='length', y='freq', alpha=.4)
```



## 읽는 법과 손질

**names** 머리글이 없는 파일이라 열 이름을 직접 준다

**sep=r'\s+'** 공백 하나 이상으로 나눈다. 앞머리 공백도 알아서 넘긴다

**alpha** 점을 반투명하게. 겹친 자리가 진해져 **밀도**가 보인다

**set\_yscale** `plt.gca().set_yscale('log')` — 세로축을 로그로

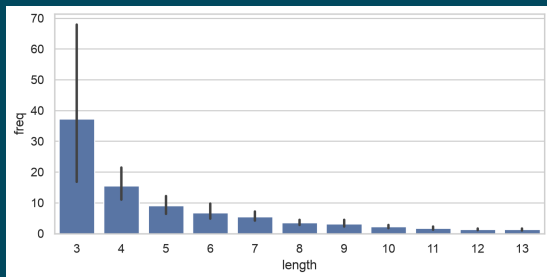
왼쪽 그림은 거의 아무 말도 하지 않는다

AND(1,265)와 THE(1,009) 둘 때문에 나머지 2,432개가 바닥에 깔려 버렸다. 로그 축으로 바꾸자 층층이 쌓인 구조가 드러난다. 5강에서  $r$ 이  $-0.14$ 에서  $-0.29$ 로 커진 것도 같은 까닭이다.

## 막대그림 — 길이별 평균 빈도

노트북 입력 — 검은 선은 seaborn이 자동으로 계산한 95% 신뢰구간이다

```
sns.barplot(wf, x='length', y='freq')
```



barplot은 y의 평균을 막대로 그린다. 3글자 37.32회, 10글자 2.22회. 숙제03의 for 문 여덟 줄이 한 줄이 되었다. 그런데 이 그림을 믿어도 될까?

# 막대가 감춘 것

3글자 막대는 37.32를 가리킨다. 그 자리에 단어가 있는가

중앙값은 **6**이고 130개 중 **112개(86%)**가 평균보다 적다. 평균은 Q3(18.75)보다도 위, 곧 자료의 4분의 3보다 위에 찍혀 있다. AND(1,265)와 THE(1,009) **두 낱말**이 한 일이다.

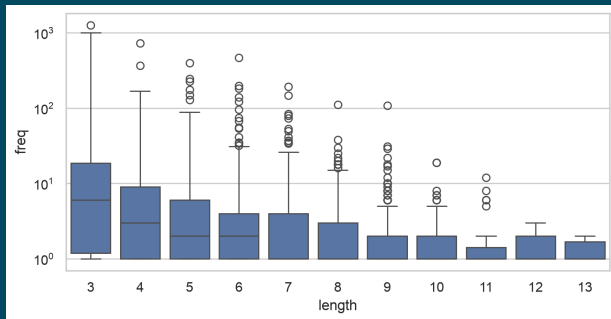
*dynamite plot* — 평균 막대에 오차막대를 얹은 그림을 부르는 이름

- 0부터 칠한 면적은 양을 뜻한다. 개수·합계에는 맞지만 평균에는 맞지 않는다.
- 관측이 몇 개인지 안 보인다. 3글자는 130개, 13글자는 **6개**인데 굵기가 같다.
- 검은 선이 표준편차인지 신뢰구간인지 모른다. 폭도 48.5부터 0.7까지 제각각이다.
- 서로 다른 분포가 **똑같은 막대**를 만든다. 요약된 두 수 말고는 남는 것이 없다.

# 상자그림으로 다시 그리면

노트북 입력 — 빈도는 한쪽으로 몰려 있으니 세로축을 로그로

```
sns.boxplot(wf, x='length', y='freq', log_scale=True)
```



중앙값이 6, 3, 2, 2, 1...로 내려간다. Zipf의 약어 법칙이 평균 막대보다 오히려 뚜렷하다.

# 그림을 고를 때의 원칙

## 막대가 맞는 자리

- 개수, 합계, 비율
- 0이 뜻을 가지는 양
- 예: 권별 레시피 편수

## 분포를 보여야 하는 자리

- 평균·중앙값 같은 요약값
- 치우친 자료, 작은 표본
- 예: 오늘의 길이별 빈도

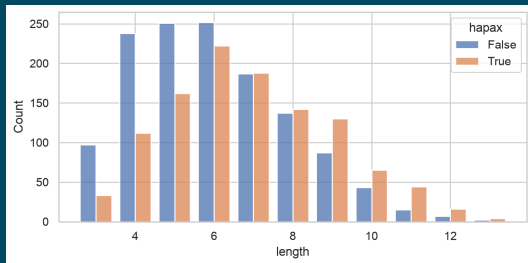
## 5강의 그 말이 그림에도 똑같이 적용된다

“요약값 하나는 반드시 무엇인가를 감춘다.” 막대그림은 그 요약값만 남기는 그림이다. 자료가 적어 다 그릴 수 있으면 다 그리고, 많으면 상자그림처럼 **퍼짐이 보이는** 그림을 쓴다.

# hue — 한 그림 안에서 갈라 보기

## 노트북 입력

```
wf['hapax'] = wf['freq'] == 1  
sns.histplot(wf, x='length', hue='hapax', discrete=True,  
             multiple='dodge', shrink=.8)
```



# Zipf의 약어 법칙을 그림으로

## 노트북 입력

```
print(wf.groupby('hapax')['length']  
      .mean().round(2))
```

## 실행 결과

```
hapax  
False    6.01  
True     6.88  
Name: length, dtype: float64
```

6강의 “4.72자 대 6.88자”가 이제 분포로 보인다

- 6글자까지는 파란 막대, 즉 **되풀이되는 말**이 더 많다.
- 7글자부터는 주황 막대, 즉 **딱 한 번 쓰인 말**이 더 많다. (7글자는 188 대 187)
- hue에 열 이름을 주면 그 값마다 색을 나누고 범례를 붙여 준다.

# 파일로 저장하기

터미널 입력 — 답을 자리를 먼저 만든다. 없으면 FileNotFoundError가 난다

```
mkdir -p plots
```

## 노트북 입력

```
1 ax = sns.histplot(df, x='words')
2 ax.set_xlabel('words per recipe')
3 ax.set_title('Apicius: recipe length')
4 plt.savefig('plots/words-hist.png', dpi=150, bbox_inches='tight')
```

**ax** seaborn이 돌려주는 **그림틀**. 여기에 대고 손질한다

**dpi** 해상도. 발표 자료나 논문에는 150 이상을 쓴다

**bbox\_inches** 'tight'로 두면 둘레의 빈 자리를 잘라 준다

노트북 안의 그림은 커널을 다시 켜면 사라진다. 남길 그림은 **파일로 저장하고 커밋한다**.

# 한글이 네모로 나올 때

## 실행 결과 — 축 이름을 한글로 적으면

```
UserWarning: Glyph 47112 (\N{HANGUL SYLLABLE RE}) missing from  
font(s) DejaVu Sans.
```

## 까닭과 대처

matplotlib의 기본 글꼴에는 한글이 없다. Codespaces에는 한글 글꼴 자체가 없다.

```
sudo apt-get install -y fonts-nanum && rm -rf ~/.cache/matplotlib  
plt.rcParams['font.family'] = 'NanumGothic'
```

당장은 축 이름을 영어로 두어도 된다. 한글 이름표가 꼭 필요해지는 것은 10강이다.

# 무엇을 남길까

터미널 입력 — 노트북이 목록에 없으면 Ctrl+S로 저장부터 한다

```
cd /workspaces/LDS2026  
git status
```

실행 결과 (book.tsv는 data/ 아래라 3강의 .gitignore에 걸린다)

```
On branch main  
Your branch is up to date with 'origin/main'.  
  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
    07-pandas.ipynb  
    awk/book.awk  
    plots/  
  
nothing added to commit but untracked files present (use "git add" to track)
```

# 커밋하기

## 터미널 입력

```
git add 07-pandas.ipynb awk/book.awk plots/  
git commit -m "7강 실습 노트북과 코드"
```

## 실행 결과

```
[main a1b2c3d] 7강 실습 노트북과 코드  
3 files changed, 1168 insertions(+)  
create mode 100644 07-pandas.ipynb  
create mode 100644 awk/book.awk  
create mode 100644 plots/words-hist.png
```

1,168줄 가운데 1,152줄이 노트북이다. 그림은 **한 줄도 세지 않는다**. PNG는 글자가 아니라서다.

# 푸시하기

## 터미널 입력

```
git push
```

## 실행 결과

```
Enumerating objects: 9, done.  
Writing objects: 100% (7/7), 167.32 KiB | 16.73 MiB/s, done.  
To https://github.com/your-username/LDS2026.git  
e4f5a6b..a1b2c3d main -> main
```

## 그림도 결과도 함께 올라간다

브라우저에서 07-pandas.ipynb를 열면 GitHub가 표와 그림까지 그려 준다. 낡은 결과가 그대로 남으므로, 올리기 전에 [Restart] 후 [Run All]로 다시 돌려 둔다.

1 데이터프레임

2 표 다루기

3 시각화

4 **마무리**

요약

다음 시간 예고

과제

# 오늘 쓴 것

| 하는 일     | 6강의 numpy                | 오늘의 pandas                  |
|----------|--------------------------|-----------------------------|
| 표 읽기     | 열마다 np.loadtxt 한 번       | pd.read_csv 한 번             |
| 열 부르기    | 배열 이름을 따로 기억             | df['words']                 |
| 기술통계량    | .mean(), .std(ddof=1)... | .describe()                 |
| 조건에 맞는 행 | 마스크를 배열마다 씌우기            | df[df['words'] > 200]       |
| 정렬       | 짜이 어긋난다                  | df.sort_values('words')     |
| 문자열 열    | 리스트 컴프리헨션                | df['title'].str.len()       |
| 세기·뭉기    | 마스크 .sum()               | value_counts, groupby       |
| 표 잇기     | 손으로                      | df.merge(bk, on='number')   |
| 빈 자리     | 모르고 지나간다                 | NaN, isna, fillna           |
| 그리기      | —                        | sns.histplot(df, x='words') |

도구가 바뀌어도 답은 그대로다. 바뀐 것은 **한 번에 볼 수 있는 크기**다.

# 오늘 배운 것

## 데이터프레임

- Series는 색인 붙은 배열, DataFrame은 그 모음
- info로 자료형과 빈 자리를 본다
- describe 한 줄에 5·6강의 값

## 표 다루기

- 고르기·정렬·새 열·.str
- merge로 잇고 groupby로 묶는다
- NaN은 메우기 전에 까닭을 찾는다

## 시각화

- 표를 넘기면 축과 범례는 저절로
- 분포·요약·비교·관계, 네 가지
- bins와 축 척도가 인상을 바꾼다
- 평균 막대는 퍼짐을 감춘다

## 언어데이터

- 권별 28.9단어 대 81.3단어
- 제목 116편이 ANOTHER로 시작
- 단발어는 길고 고빈도어는 짧다

## 다음 시간에 배울 것 (9월 28일 월요일)

### 제8강 코퍼스언어학: 단어 빈도와 생산성

- 오늘 `value_counts`로 센 빈도가 무엇을 재는 값인지 따져 본다.
- 토큰과 타입, 그리고 코퍼스를 키우면 어휘가 어떻게 늘어나는가.
- 6강에서 만난 **단발어** 1,118개가 생산성 측정의 재료가 된다.

### 오늘 이후로는

4강부터 오늘까지 AWK와 numpy와 pandas로 **같은 요리책**을 세 번 분석했다. 도구 이야기는 여기서 한 매듭을 짓고, 다음 두 강의는 **무엇을 세고 있었는가**를 묻는다.

## [숙제04] seaborn 연습 (4점)

### 할 일

노트북 hw04.ipynb를 만들어 아래 넷을 푼다. 각 문항 1점.

- 1 index.tsv와 book.tsv를 붙인 표에서 권별 **레시피 편수**를 막대그림으로 그린다.
- 2 권별 낱말 수의 **중앙값**이 가장 큰 권과 가장 작은 권을 코드로 찾아 출력한다.
- 3 word-freq.txt에서 빈도 상위 20개 단어를 가로 막대그림으로 그린다.
- 4 제목이 ANOTHER로 시작하는 레시피와 그렇지 않은 레시피의 길이 분포를 hue로 갈라 한 그림에 그리고, 무엇이 보이는지 두 문장으로 적는다.

### 제출

기한 **2026-10-02 12:00 (금요일 정오)**. hw04.ipynb를 저장소에 push한다. 그림에는 **제목과 축 이름**을 붙인다. data/는 커밋하지 않는다.

## [숙제04] 힌트

### 힌트

- 1 `sns.countplot(df, y='book')`이면 한 줄이다. `order`를 잊지 않는다.
- 2 `g = df.groupby('book')['words'].median()` 뒤에 `g.idxmax()`, `g.idxmin()`. 답을 손으로 적지 않는다.
- 3 `wf.nlargest(20, 'freq')`로 표를 먼저 만들고 `sns.barplot`에 넘긴다.
- 4 `df['another'] = df['first'] == 'ANOTHER'`로 열을 만든다.

**4번을 위한 참고:** ANOTHER로 시작하는 116편은 앞 레시피의 변형이다. 그렇다면 길어도 짧을 것 같은가? 그런 다음 `groupby`로 확인해 보자.

제출 전 [Restart] 후 [Run All]. 질문은 eTL Q&A 게시판에. (답변 마감: 목요일 12:00)